CASINO

Se desea desarrollar una aplicación Java que simule el funcionamiento de una ruleta francesa y diferentes estrategias de juego.

La ruleta dispone de **37 números, del 0 al 36**. El programa realizará una tirada cada **3000 milisegundos**.

En la aplicación participarán varios jugadores que ejecutarán su estrategia de forma concurrente. Todos los jugadores comienzan con un saldo de **1.000 €** y la banca dispone inicialmente de **50.000 €**.

La aplicación deberá incluir las siguientes estrategias.

Apuesta a un número

Habrá **4 jugadores** que elegirán aleatoriamente un número entre 1 y 36. En cada turno apostarán 10 € a ese número

- si pierden, se descontarán los 10 € de su saldo
- si ganan, recibirán 360 €
- el número 0 no podrá ser elegido como número apostado.

Apuesta a par o impar

Habrá **4 jugadores** que elegirán aleatoriamente entre par e impar. En cada turno apostarán 10 €

- si pierden, se descontarán los 10 €
- si ganan, recibirán 20 €

Estrategia de Martingala

Habrá **4 jugadores** que utilizarán la estrategia de martingala. Cada jugador elegirá un número entre 1 y 36 y comenzará apostando 10 €.

- Si pierde, duplicará la cantidad apostada en el siguiente turno
  (e. 10 → 20 → 40 → 80 → ...)
- Si gana, recibirá el premio correspondiente y deberá continuar aplicando la estrategia establecida.

Funcionamiento de la banca

- La banca deberá aceptar las apuestas y realizar los pagos correspondientes.
- La banca **nunca podrá pagar una cantidad superior al saldo disponible**.
- Si el número ganador es **0**, todas las apuestas perderán y la banca conservará las cantidades apostadas.

Funcionamiento concurrente

- Los jugadores deberán ejecutarse mediante **hilos independientes**.

- Todos los jugadores utilizarán la **misma ruleta**.
- La aplicación deberá coordinar los turnos de juego de manera que:
  - Los jugadores realicen sus apuestas.
  - Los jugadores esperen hasta conocer el resultado de la tirada.
  - La ruleta genere el número ganador.
  - Cada jugador compruebe el resultado de su apuesta.
  - Se notifique a los jugadores que puede comenzar un nuevo turno.
  - El proceso continúe mientras la simulación permanezca activa.

Orientación técnica

La solución deberá utilizar los mecanismos de sincronización estudiados en la unidad. Se recomienda utilizar un objeto compartido que actúe como **monitor** para coordinar los jugadores y la ruleta. Deberán utilizarse:

clases que hereden de Thread, synchronized, wait(), notifyAll(), recursos compartidos, coordinación entre los diferentes hilos.

No se permite utilizar mecanismos de sincronización no estudiados en la unidad, como Lock, ReentrantLock o Semaphore.

Requisitos mínimos

La aplicación deberá crear los **12 jugadores**;

utilizar una única ruleta compartida;

utilizar una única banca compartida;

ejecutar las tres estrategias;

mostrar por pantalla las tiradas realizadas;

mostrar las apuestas realizadas;

mostrar los resultados de las apuestas;

mostrar la evolución de los saldos;

garantizar que los accesos concurrentes a los recursos compartidos sean seguros;

coordinar correctamente los diferentes turnos;

finalizar correctamente la ejecución cuando la banca no pueda continuar pagando las apuestas.

Resultado esperado

La ejecución deberá permitir observar claramente la evolución de la simulación. Por ejemplo:

```
Nueva tirada: 17
JugadorNumero 1 apuesta 10 € al número 12
JugadorNumero 2 apuesta 10 € al número 17
JugadorParImpar 5 apuesta 10 € a PAR
```

```
JugadorMartingala 9 apuesta 10 € al número 23
```

Resultado de la tirada: 17

```
JugadorNumero 1 pierde
JugadorNumero 2 gana
JugadorParImpar 5 pierde
JugadorMartingala 9 pierde
```

Los mensajes concretos y su orden podrán variar debido a la ejecución concurrente.

Entrega

La entrega deberá realizarse en **un único archivo PDF** que documente la realización de la actividad. El documento deberá incluir:

- **Caso práctico realizado**, acompañado de capturas del código fuente relevante correspondiente.
- **Evidencias de realización** mediante capturas de pantalla en las que sea posible comprobar, cuando corresponda, el **usuario, la fecha y la hora de ejecución**.
- **Una breve justificación o explicación** de los resultados obtenidos.
- Cuando se utilicen **funciones, métodos, clases, estructuras o recursos que no hayan sido explicados o trabajados**, se deberá incluir una justificación de su utilización, explicando **qué hacen, por qué son necesarios y cómo se han utilizado** en la solución.
- **Enlace al código fuente en GitHub**.

Criterios de evaluación

Criterio	Excelente	Adecuado	Insuficiente
Realización y resultado (40%)	Implementa correctamente la simulación, las estrategias de juego y la coordinación de los hilos, obteniendo los resultados esperados.	La aplicación funciona en su mayor parte, aunque presenta algún error menor o alguna funcionalidad incompleta.	Presenta errores importantes de funcionamiento, problemas de sincronización o deja partes relevantes sin realizar.

Comprensión y justificación (25%)	Explica correctamente el funcionamiento de la aplicación, la sincronización utilizada y los resultados obtenidos, demostrando comprensión de la solución.	La explicación es correcta, aunque superficial o presenta algún error menor.	No justifica adecuadamente la solución o demuestra escasa comprensión del funcionamiento de los hilos y la sincronización.
Evidencias (25%)	Aporta evidencias suficientes, claras y vinculadas al trabajo realizado; permiten comprobar el funcionamiento de la aplicación y la realización de la práctica.	Aporta evidencias, pero alguna es insuficiente, poco clara o no permite comprobar completamente algún aspecto.	Las evidencias son insuficientes o no permiten comprobar adecuadamente la realización de la práctica.
Entrega y presentación (10%)	Entrega completa, ordenada y siguiendo todas las indicaciones establecidas.	Presenta alguna deficiencia menor en la organización o en el cumplimiento de las indicaciones.	Entrega incompleta, desordenada o no sigue las indicaciones establecidas.